



**Universidad
Gerardo Barrios**

Nombre del proyecto:

ReceiptIA

Módulo:

Desarrollo de Aplicaciones con IA

Integrantes:

Jonathan Elías Gámez Larin

Wilber José Jiménez Ramírez

David Arnoldo Hernández Gómez

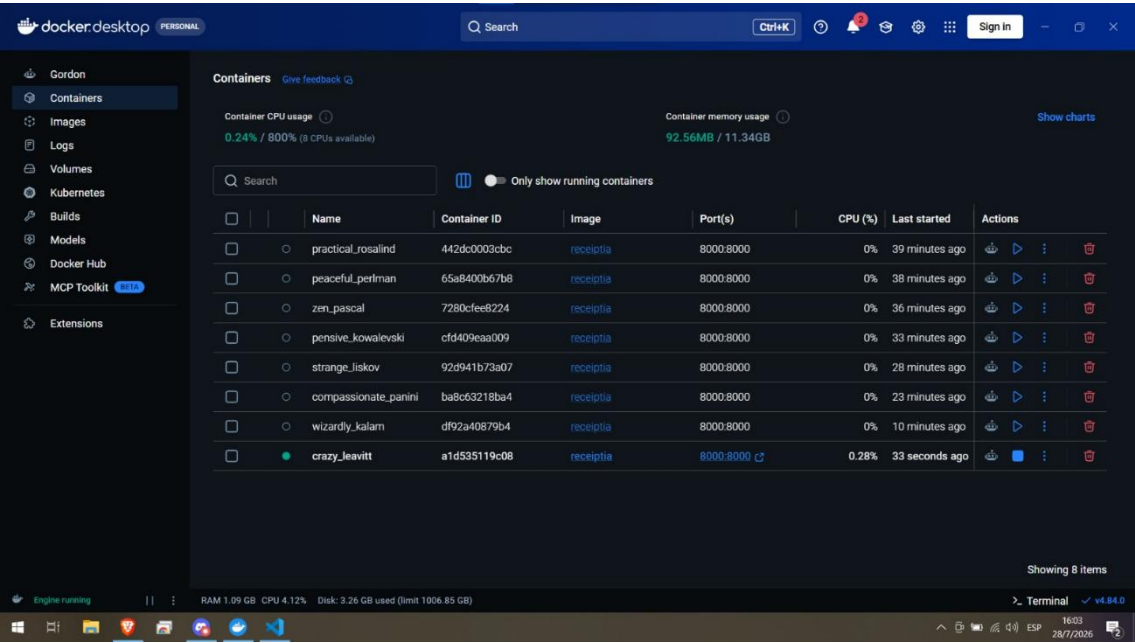
Fecha de entrega:

01/08/26

Descripción breve del servicio preparado

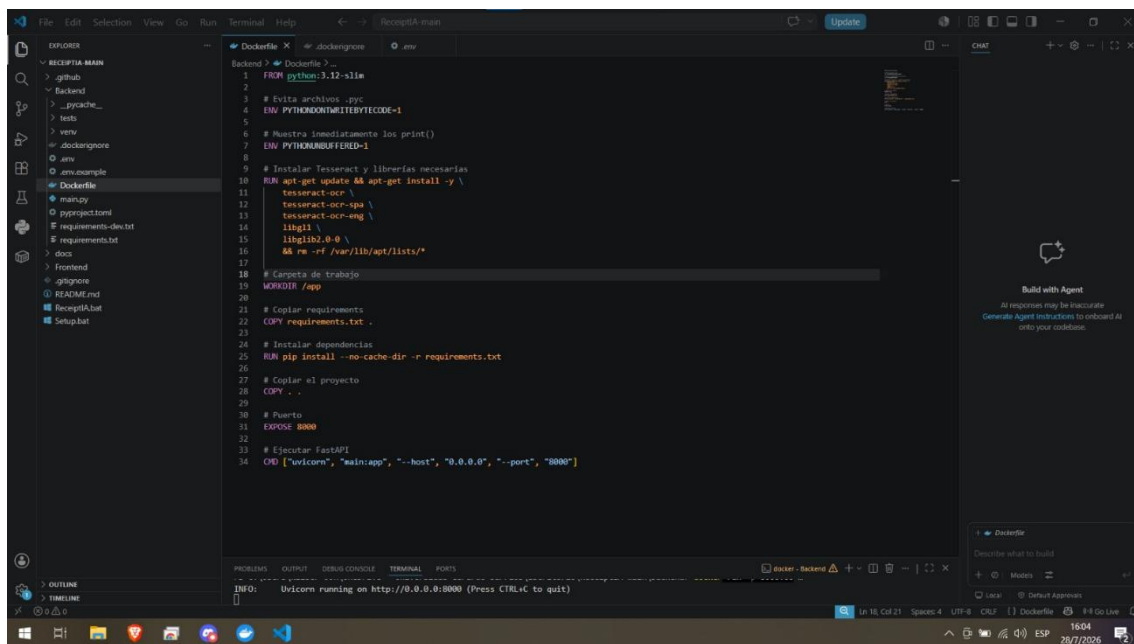
ReceiptIA es una API desarrollada con FastAPI que permite el procesar facturas y recibos utilizando OCR(Tesseract) e inteligencia artificial (Gemini) para lograr extraer la información, detectar anomalías y devolver los datos estructurados en formato JSON.

Ruta elegida



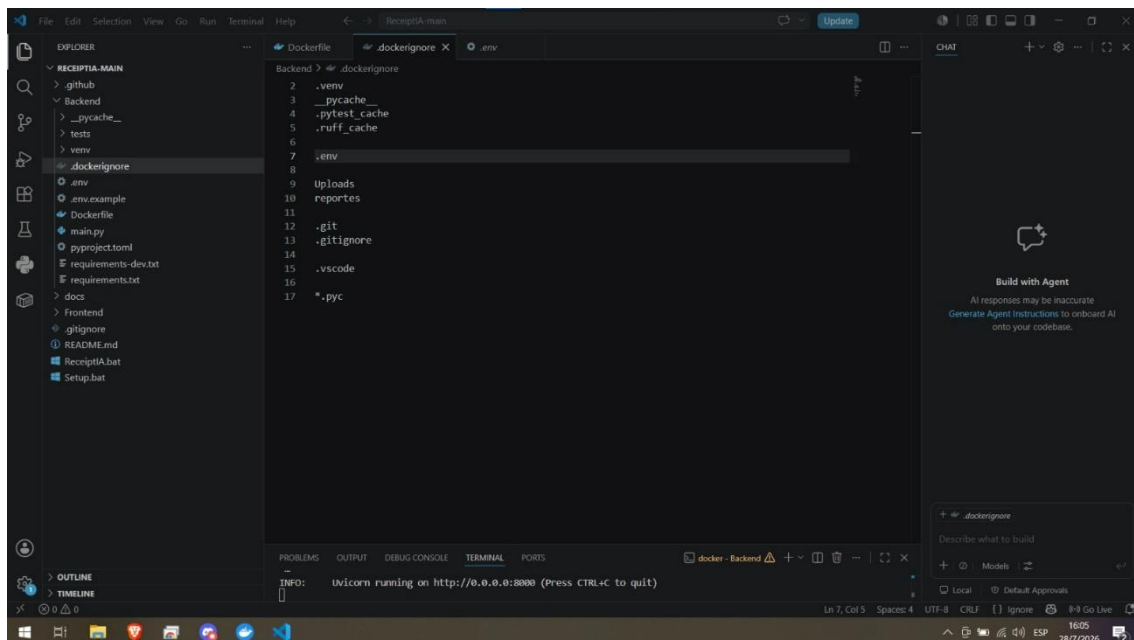
El contenedor de ReceiptIA se está ejecutando de forma correcta en Docker Desktop, se verifica que este activo ya que se aprecia el puerto 8000 lo que confirma que la API esta lista para recibir solicitudes y procesar los servicios de backend.

Dockerfile



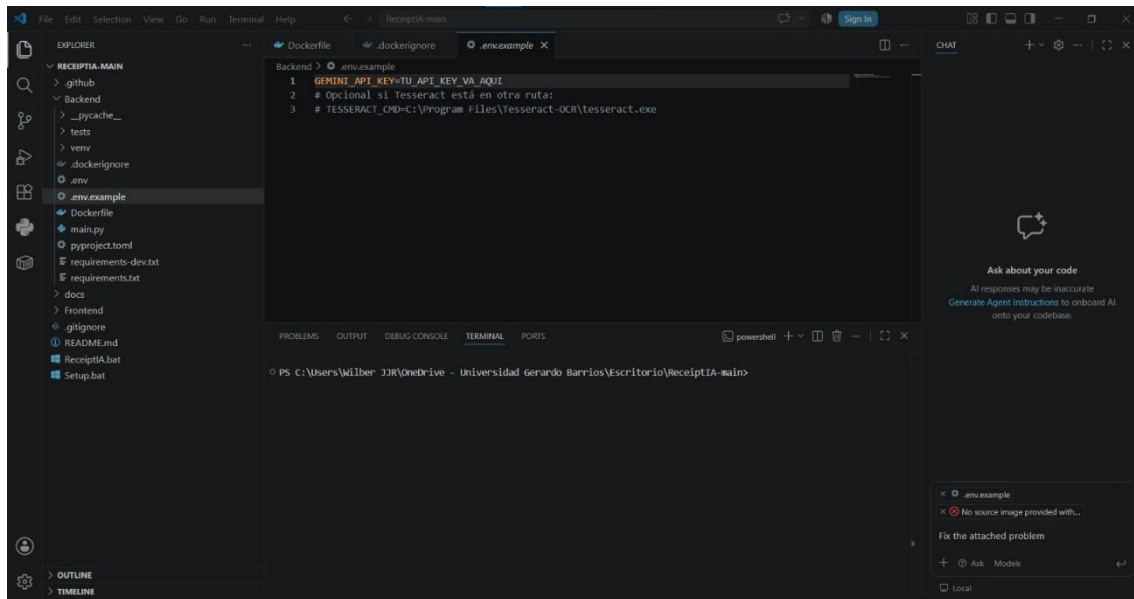
El archivo Dockerfile se define en base de Python, la instalación de las dependencias del sistema y del mismo Python también la configuración del directorio de trabajo, la exposición del puerto 8000 y el comando de inicio de la aplicación por medio de Uvicorn permitiendo la ejecución de la API dentro del contenedor Docker

.dockerignore



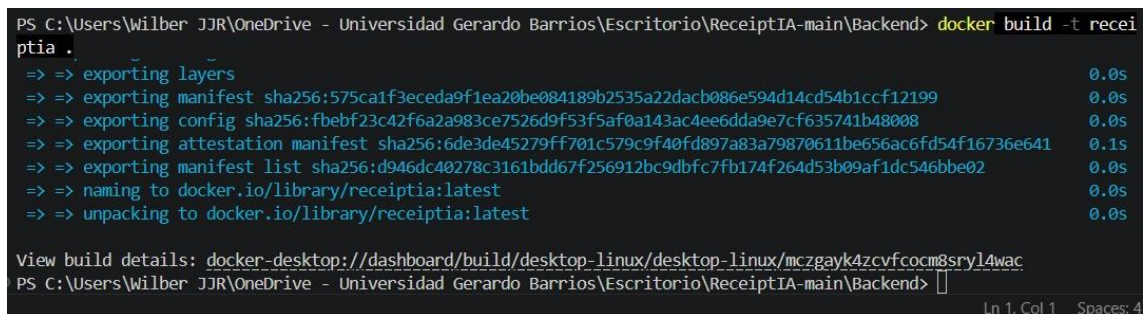
.dockerignore se utiliza para excluir archivos y carpetas innecesarios durante la construcción de a imagen Docker permitiendo reducir el tamaño del contenedor, evitando incluir información sensible como la que contiene el archivo .env y optimiza el proceso de despliegue de la aplicación.

Variables de entorno

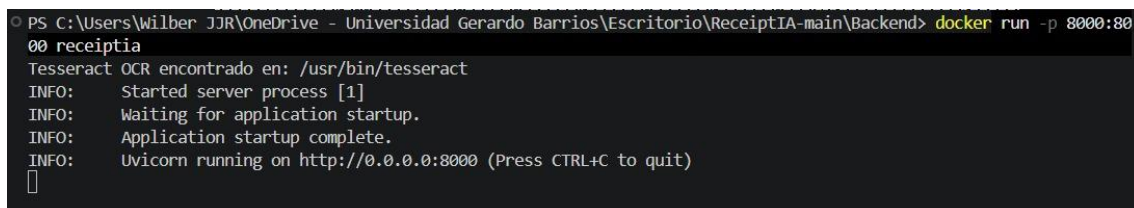


.env.example proporciona un ejemplo de las variables de entorno necesarias para ejecutar la aplicación como lo es la clave API de Gemini y la configuración opcional de Tesseract OCR, este archivo sirve como plantilla para que cada usuario configure su propio archivo .env sin llegar a exponer la información confidencial.

Evidencia de construcción, ejecución o despliegue

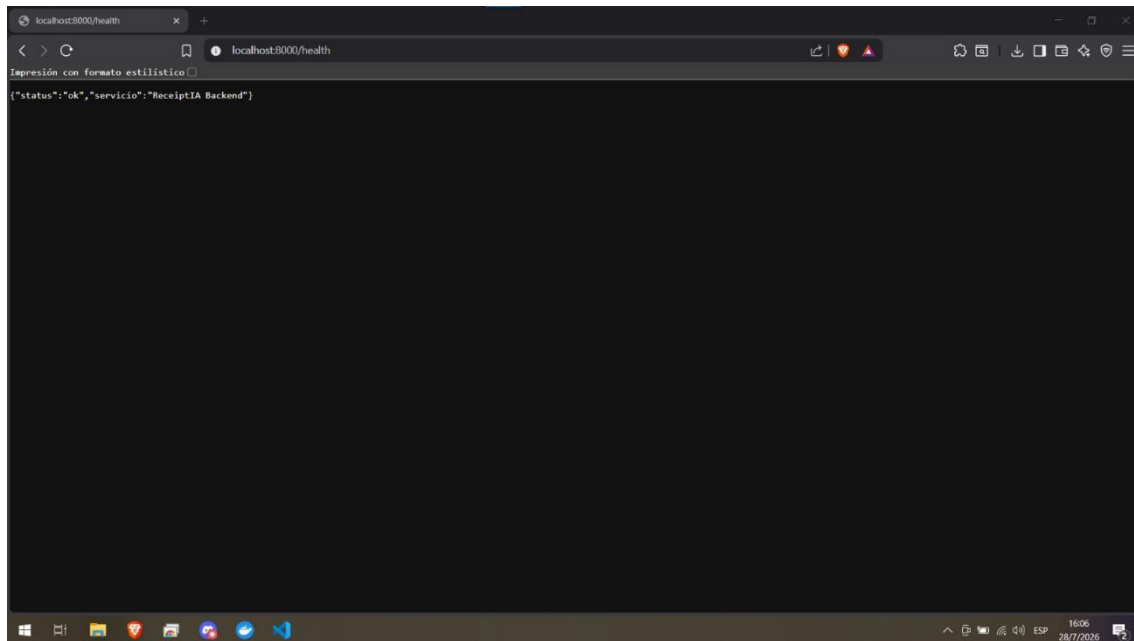


Se muestra la ejecución del comando “docker build -t receiptia” mediante el cual se construye correctamente la imagen Docker del proyecto, el proceso finaliza sin errores y genera la imagen “receiptia:latest” para ser utilizada en la creación y en la ejecución de contenedores



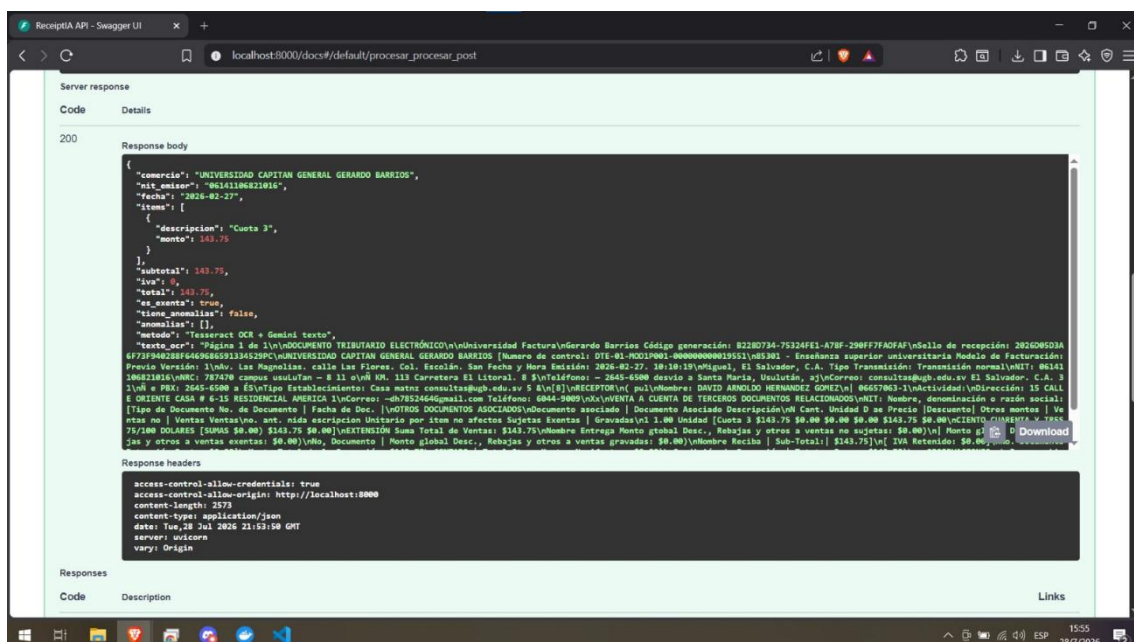
La ejecución del comando “docker run -p 8000:8000 receiptia” mediante el cual inicia un contenedor basado en la imagen creada anteriormente, el registro confirma que Tesseract OCR fue detectado de forma correcta y que el servicio Uvicorn inicio sin errores quedando disponible para el puerto 8000 para atender las solicitudes de la API

Prueba del endpoint /health



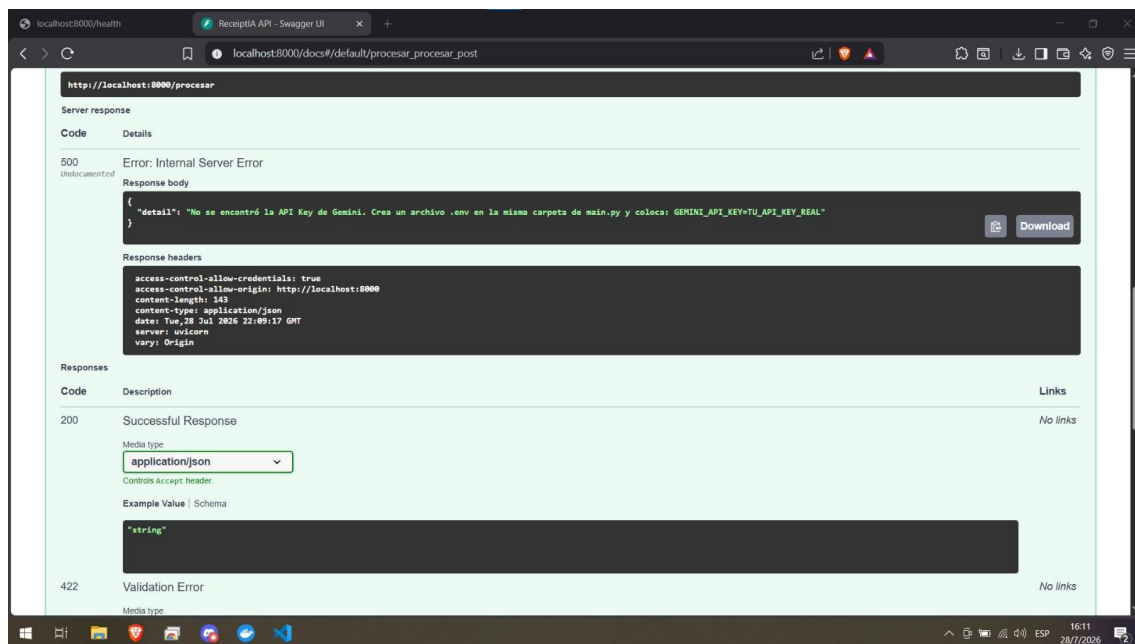
El endpoint `/health` se está ejecutando desde el contenedor de Docker la respuesta `{"status":"ok","servicio":"ReceiptIA Backend"}` confirma que la API se encuentra activa y funciona correctamente

Prueba del endpoint principal del proyecto.



Observamos el resultado obtenido tras ejecutar el endpoint /procesar, la API responde correctamente con un código 200 (Ok) y devuelve la información extraída de la factura, incluyendo datos como el comercio, fechas, subtotal, total anomalías detectadas y el texto reconocido por OCR y el procesamiento con inteligencia artificial

Errores, intentos, bloqueos y correcciones realizadas.



El error detectado al ejecutar el endpoint /procesar debido a la ausencia de la variable GEMINI_API_KEY, se identificó el problema y se corrigió creando el archivo .env con la información requerida.

Plan de infraestructura mínima

El proyecto utiliza una arquitectura sencilla basada en contenedores para facilitar su despliegue y portabilidad.

Infraestructura:

- Aplicación Backend desarrollada con **FastAPI**.
- Servidor de aplicaciones **Uvicorn**.
- Contenedor **Docker** para la ejecución del sistema.
- Variables de entorno mediante archivo **.env** para proteger la API Key de Gemini.
- Integración continua mediante **GitHub Actions** para validar automáticamente el código y ejecutar pruebas.
- Puerto de exposición del servicio: **8000**.

Estimación de costos iniciales con supuestos

Para la etapa de desarrollo se consideran los siguientes supuestos:

Recurso	Costo estimado
GitHub (repositorio público)	Gratuito
GitHub Actions	Gratuito (límite incluido)
Docker Desktop (uso académico)	Gratuito
FastAPI	Gratuito
Python	Gratuito
Tesseract OCR	Gratuito
Gemini API (plan gratuito)	Gratuito mientras no se excedan los límites
Equipo local del desarrollador	Sin costo adicional

Costo estimado inicial: US\$0, asumiendo el uso de los planes gratuitos y la ejecución del sistema en un equipo local.

Riesgos técnicos pendientes

- Dependencia de la disponibilidad de la API de Gemini.
- Posibles cambios en los límites gratuitos del servicio de IA.
- Disminución de precisión del OCR cuando las facturas poseen baja calidad o imágenes borrosas.
- Posibles variaciones en el formato de facturas de diferentes emisores.
- Necesidad de fortalecer la seguridad y autenticación en caso de desplegar el sistema en producción.
- Optimización del rendimiento para procesamiento de múltiples documentos simultáneamente

Repositorio actualizado del proyecto

<https://github.com/DavidHernandezd/ReceiptIA.git>